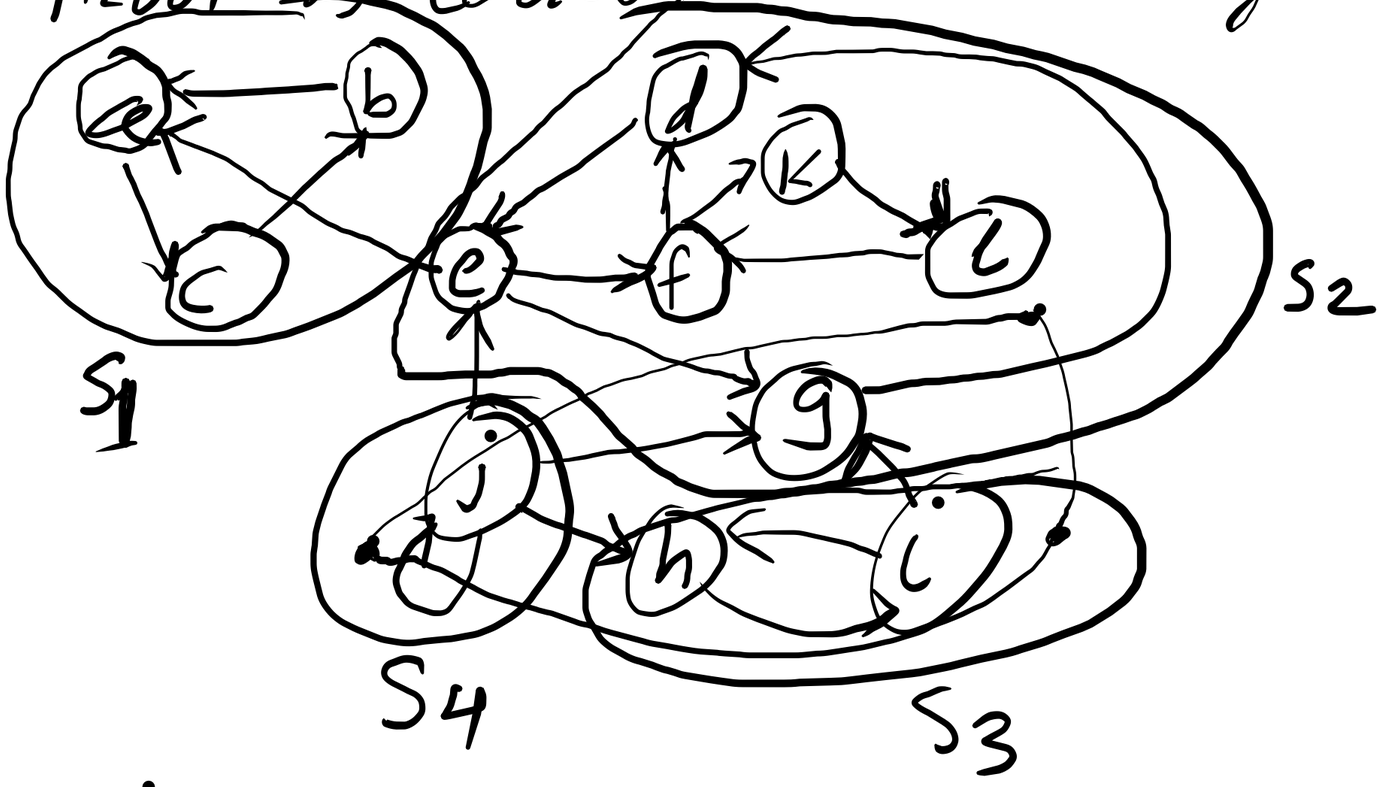
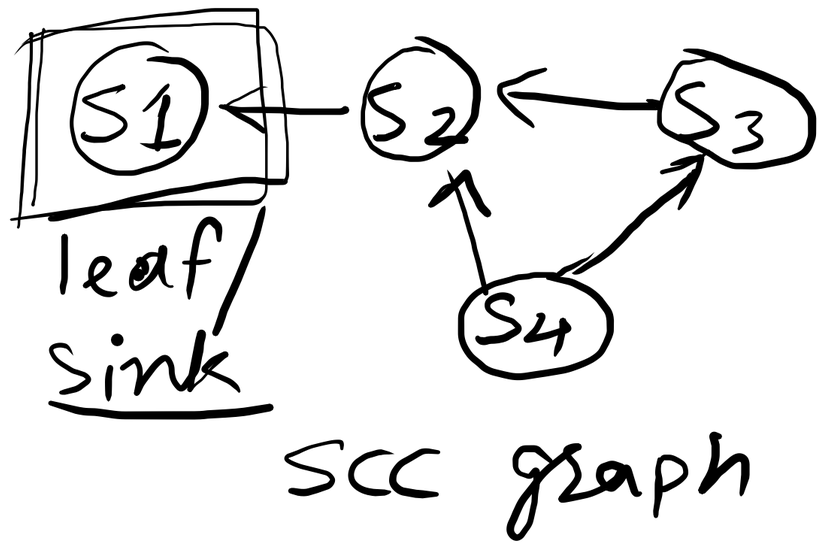
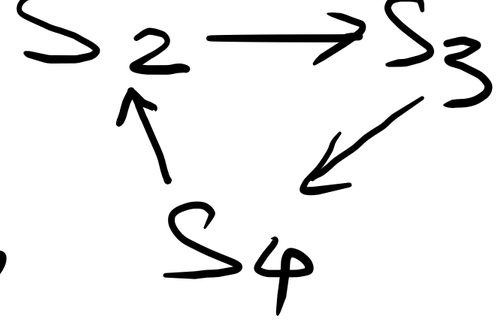
Proof of correctness: Finding SCCs







if there is vertex v in S_j & S_i then from any vertex in S_j we can go to any vertex in S_i through v and vice-a-verso. which violates minimality property

Define scc as a sink if it has no outgoing edges in G^{scc}

Lemma: There must be atlease one sink scc in G^{scc}

Proof: Since G^{scc} is a DAG, it follows topological order. The last vertex of the topological order cannot have outgoing edges.

DFS in sink scc:

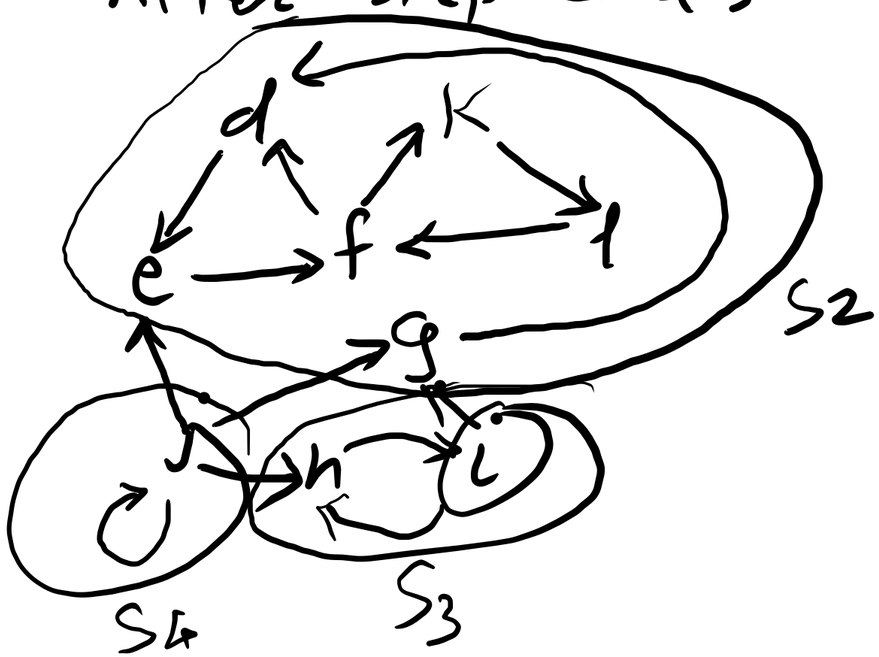
let s be a sink scc in G^{scc}. When we perform DFS starting from any vertex in sink scc, then it will output the tree that must include all and only vertices in Sink scc.

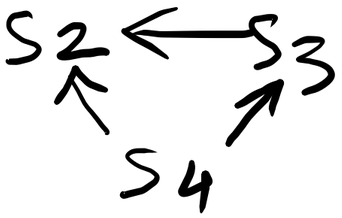
Strategy to find SCC:

1 Perform DFS from any vertex in sink SCC
2 Delete all vertices from G including edges.
3 Delete sink SCC from G^{SCC}
4 Repeat step 1 - 3

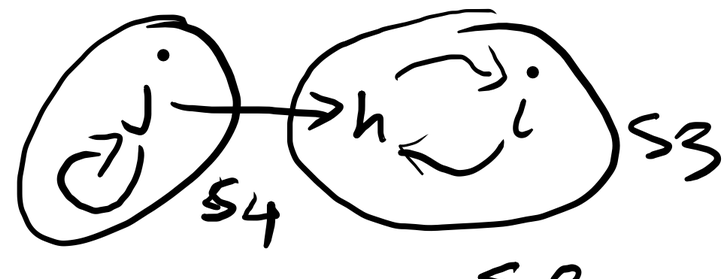
Run DFS in S_1 sink vertex
$\{a, b, c\}$ will be discovered

After step 2 & 3





Run DFS in S_2 and discover $\{d, e, f, k, l, g\}$
After step 2 &



Run DFS in S_3
discover $\{h, i\}$
After step 2 & 3

Last iteration
Run DFS in S_4

$$L = \{ \underset{\uparrow}{a}\ b\ c\ \underset{\uparrow}{f}\ e\ d\ g\ \underset{\uparrow}{i}\ h\ \underset{\uparrow}{j}\ k\ l \}$$

S1 S2 S3 S4

Bellman-Ford Algorithm:

- Network Routing
- GPS : weights can be define.
  - weight is time, Fastest Route
  - weight is distance, Shortest Route
  - weight is cost, lowest cost Route

Shortest path problem: